PROJECT 1:LIBRARY B00K MANAGEMENT:

LibraryBookManagementApplication.java

```
package com.example.library;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class LibraryBookManagementApplication {
   public static void main(String[] args) {
      SpringApplication.run(LibraryBookManagementApplication.class, args);
   }
}
```

Book.java

```
package com.example.library.model;
import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import java.math.BigDecimal;
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Book {
private Long id;
@NotBlank(message = "Book name is required")
```

```
@Size(min = 3, max = 50, message = "Book name must contain 3 to 50 characters")
private String name;
@NotBlank(message = "Author name is required")
private String author;
@NotNull(message = "Price is required")
@DecimalMin(value = "0.0",inclusive = false, message = "Price must be greater than zero")
private BigDecimal price;
@NotNull(message = "Available quantity is required")
@Min(value = 0, message = "Quantity cannot be negative")
private Integer quantity;
}
```

Bookservice.java

```
package com.example.library.service;
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import org.springframework.stereotype.Service;
import com.example.library.exception.BookNotFoundException;
import com.example.library.model.Book;
@Service
public class BookService {
private final ArrayList<Book> books = new ArrayList<>();
private long nextId = 1;
// Add Book
public Book addBook(Book request) {
```

```
            if (request.getPrice().compareTo(BigDecimal.ZERO) <= 0) {
                throw new IllegalArgumentException("Price must be greater than zero");
            }

    Book book = new Book(
            nextId++,
            request.getName().trim(),
            request.getAuthor().trim(),
            request.getPrice(),
            request.getQuantity()
    );
    books.add(book);
    return book;
}
// Get All Books
public List<Book> getAllBooks() {
    return books.stream()
            .map(book -> new Book(
                    book.getId(),
                    book.getName(),
                    book.getAuthor(),
                    book.getPrice(),
                    book.getQuantity()))
            .toList();
}
// Get Book By ID
public Book getBookById(Long id) {
```

```
    return books.stream()
            .filter(book -> book.getId().equals(id))
            .findFirst()
            .map(book -> new Book(
                    book.getId(),
                    book.getName(),
                    book.getAuthor(),
                    book.getPrice(),
                    book.getQuantity()))
            .orElseThrow(() ->
                    new BookNotFoundException("Book not found with ID: " + id));
}
// Update Book
public Book updateBook(Long id, Book request) {
    Book existing = findBook(id);
    existing.setName(request.getName().trim());
    existing.setAuthor(request.getAuthor().trim());
    existing.setPrice(request.getPrice());
    existing.setQuantity(request.getQuantity());
    return getBookById(id);
}
// Delete Book
public String deleteBook(Long id) {
    Book book = findBook(id);
    books.remove(book);
    return "Book deleted successfully";
}
// Partial Update
```

```
public Book partialUpdateBook(Long id, Book request) {
   Book existing = findBook(id);
   if (request.getName() != null && !request.getName().isBlank()) {
      existing.setName(request.getName().trim());
   }
   if (request.getAuthor() != null && !request.getAuthor().isBlank()) {
      existing.setAuthor(request.getAuthor().trim());
   }

   if (request.getPrice() != null) {
      if (request.getPrice().compareTo(BigDecimal.ZERO) <= 0) {
         throw new IllegalArgumentException(
               "Price must be greater than zero");
      }
      existing.setPrice(request.getPrice());
   }

   if (request.getQuantity() != null) {
      if (request.getQuantity() < 0) {
         throw new IllegalArgumentException(
               "Quantity cannot be negative");
      }
      existing.setQuantity(request.getQuantity());
   }

   return getBookById(id);
}
// Find Book
```

```
private Book findBook(Long id) {
   return books.stream()
         .filter(book -> book.getId().equals(id))
         .findFirst()
         .orElseThrow(() ->
               new BookNotFoundException("Book not found with ID: " + id));
}
// Custom Method 1: Search by Book Name (Partial Match)
public List<Book> searchByName(String keyword) {
   if (keyword == null || keyword.isBlank()) {
      throw new IllegalArgumentException("Search keyword must not be blank");
   }
   String searchText = keyword.trim().toLowerCase(Locale.ROOT);
   return books.stream()
         .filter(book -> book.getName()
               .toLowerCase(Locale.ROOT)
               .contains(searchText))
         .toList();
}
// Custom Method 2: Filter by Author (Full Match)
public List<Book> getByAuthor(String author) {
   if (author == null || author.isBlank()) {
      throw new IllegalArgumentException("Author name must not be blank");
   }
   return books.stream()
         .filter(book -> book.getAuthor()
               .equalsIgnoreCase(author.trim()))
         .toList();
```

```
}
// Custom Method 3: Filter Books by Maximum Price
public List<Book> getBooksWithinBudget(BigDecimal maxPrice) {
   if (maxPrice == null || maxPrice.signum() < 0) {
      throw new IllegalArgumentException(
            "Maximum price is required and cannot be negative");
   }
   return books.stream()
         .filter(book -> book.getPrice().compareTo(maxPrice) <= 0)
         .sorted(Comparator.comparing(Book::getPrice))
         .toList();
}
// Custom Method 4: Get Available Books
public List<Book> getAvailableBooks() {
   return books.stream()
         .filter(book -> book.getQuantity() > 0)
         .sorted(Comparator.comparing(
               Book::getName, String.CASE_INSENSITIVE_ORDER))
         .toList();
}
}
```

Bookcontroller.java

```
package com.example.library.controller;
import java.math.BigDecimal;
import java.util.List;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import com.example.library.model.Book;
```

```
import com.example.library.service.BookService;
@RestController
@RequestMapping("/books")
public class BookController {
private final BookService service;
public BookController(BookService service) {
   this.service = service;
}
// POST - Add Book
@PostMapping
public Book addBook(@Valid @RequestBody Book book) {
   return service.addBook(book);
}
// GET - Get All Books
@GetMapping
public List<Book> getAllBooks() {
   return service.getAllBooks();
}
// GET - Get Book By ID
@GetMapping("/{id}")
public Book getBookById(@PathVariable Long id) {
   return service.getBookById(id);
}
// PUT - Update Entire Book
@PutMapping("/{id}")
public Book updateBook(
      @PathVariable Long id,
      @Valid @RequestBody Book book) {
```

```
    return service.updateBook(id, book);
}
// PATCH - Partial Update
@PatchMapping("/{id}")
public Book partialUpdateBook(
        @PathVariable Long id,
        @RequestBody Book book) {
    return service.partialUpdateBook(id, book);
}
// DELETE - Delete Book
@DeleteMapping("/{id}")
public String deleteBook(@PathVariable Long id) {
    return service.deleteBook(id);
}
// Custom API 1 - Search by Book Name
@GetMapping("/search")
public List<Book> searchByName(@RequestParam String keyword) {
    return service.searchByName(keyword);
}
// Custom API 2 - Filter by Author
@GetMapping("/author")
public List<Book> getByAuthor(@RequestParam String author) {
    return service.getByAuthor(author);
}
// Custom API 3 - Filter by Maximum Price
@GetMapping("/budget")
public List<Book> getBooksWithinBudget(
        @RequestParam BigDecimal maxPrice) {
```

```
    return service.getBooksWithinBudget(maxPrice);
}
// Custom API 4 - Get Available Books
@GetMapping("/available")
public List<Book> getAvailableBooks() {
    return service.getAvailableBooks();
}
}
```

BookNotFoundException.java

```
package com.example.library.exception;
public class BookNotFoundException extends RuntimeException {
public BookNotFoundException(String message) {
    super(message);
}
}
```

GlobalExceptionHandler.java

```
package com.example.library.exception;
import java.util.LinkedHashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;
@RestControllerAdvice
public class GlobalExceptionHandler {
// Custom Exception
```

```
@ExceptionHandler(BookNotFoundException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public Map<String, String> handleBookNotFound(
      BookNotFoundException ex) {
   return Map.of("message", ex.getMessage());
}
// Validation Exception
@ExceptionHandler(MethodArgumentNotValidException.class)
@ResponseStatus(HttpStatus.BAD_REQUEST)
public Map<String, String> handleValidation(
      MethodArgumentNotValidException ex) {
   Map<String, String> errors = new LinkedHashMap<>();
   ex.getBindingResult().getFieldErrors().forEach(error ->
      errors.putIfAbsent(
         error.getField(),
         error.getDefaultMessage()
      )
   );
   return errors;
}
// Illegal Argument Exception
@ExceptionHandler(IllegalArgumentException.class)
@ResponseStatus(HttpStatus.BAD_REQUEST)
public Map<String, String> handleIllegalArgument(
      IllegalArgumentException ex) {
   return Map.of("message", ex.getMessage());
}
// General Exception
```

```
@ExceptionHandler(Exception.class)
@ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
public Map<String, String> handleGeneral(Exception ex) {
   return Map.of(
      "message",
      "An unexpected error occurred"
   );
}
}
```

PROJECT 2: STUDENT COURSE REGISTRATION

StudentCourseRegistrationApplication.java

```
package com.example.student;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class StudentCourseRegistrationApplication {

	public static void main(String[] args) {

		SpringApplication.run(StudentCourseRegistrationApplication.class,
args);

	}

}
```

Student.java

```
package com.example.student.model;

import jakarta.validation.constraints.Email;

import jakarta.validation.constraints.Min;

import jakarta.validation.constraints.NotBlank;

import jakarta.validation.constraints.NotNull;

import jakarta.validation.constraints.Size;

import lombok.AllArgsConstructor;

import lombok.Data;

import lombok.NoArgsConstructor;

@Data

@NoArgsConstructor

@AllArgsConstructor

public class Student {

  private Long studentId;

  @NotBlank(message = "Student name is required")
```

```
    @Size(min = 3, max = 25, message = "Student name must contain 3 to 25 characters")
    private String studentName;
    @NotBlank(message = "Course name is required")
    private String courseName;
    @NotBlank(message = "Email is required")
    @Email(message = "Enter a valid email address")
    private String email;
    @NotNull(message = "Student age is required")
    @Min(value = 18, message = "Student age must be at least 18")
    private Integer age;
}
```

StudentService.java

```
package com.example.student.service;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import org.springframework.stereotype.Service;
import com.example.student.model.Student;
import com.example.student.exception.StudentNotFoundException;
@Service
public class StudentService {
    private List<Student> students = new ArrayList<>();
    private Long nextId = 1L;
    // POST - Register Student
    public Student registerStudent(Student student) {
        student.setStudentId(nextId++);
        students.add(student);
        return student;
```

```
}
// GET - View All Students
public List<Student> getAllStudents() {
   return students;
}
// GET - Find Student By ID
public Student getStudentById(Long id) {
   if (id == null || id <= 0) {
      throw new IllegalArgumentException("Student ID must be greater than zero");
   }
   return students.stream()
         .filter(s -> s.getStudentId().equals(id))
         .findFirst()
         .orElseThrow(() ->
            new StudentNotFoundException(
               "Student not found with ID: " + id));
}
// PUT - Update Complete Student Details
public Student updateStudent(Long id, Student newStudent) {
   Student oldStudent = getStudentById(id);
   oldStudent.setStudentName(newStudent.getStudentName());
   oldStudent.setCourseName(newStudent.getCourseName());
   oldStudent.setEmail(newStudent.getEmail());
   oldStudent.setAge(newStudent.getAge());
   return oldStudent;
}
// DELETE - Delete Student
public String deleteStudent(Long id) {
```

```
    Student student = getStudentById(id);
    students.remove(student);
    return "Student deleted successfully";
}
// PATCH - Update Selected Fields
public Student partialUpdate(Long id, Student newStudent) {
    Student oldStudent = getStudentById(id);
    if (newStudent.getStudentName() != null) {
        oldStudent.setStudentName(newStudent.getStudentName());
    }
    if (newStudent.getCourseName() != null) {
        oldStudent.setCourseName(newStudent.getCourseName());
    }
    if (newStudent.getEmail() != null) {
        oldStudent.setEmail(newStudent.getEmail());
    }
    if (newStudent.getAge() != null) {
        oldStudent.setAge(newStudent.getAge());
    }
    return oldStudent;
}
// Custom Method - Search Student By Name
public List<Student> searchByName(String name) {
    return students.stream()
            .filter(s -> s.getStudentName()
                    .toLowerCase()
                    .contains(name.toLowerCase()))
            .toList();
```

```
    }
    // Custom Method - Sort Students By Name
    public List<Student> sortByName() {
        return students.stream()
                .sorted(Comparator.comparing(Student::getStudentName))
                .toList();
    }
    // Custom Method - Get Student Names Using map()
    public List<String> getStudentNames() {
        return students.stream()
                .map(Student::getStudentName)
                .toList();
    }
    // Custom Method - Find Students by Minimum Age
    public List<Student> getStudentsByMinimumAge(int age) {
        return students.stream()
                .filter(s -> s.getAge() != null && s.getAge() >= age)
                .toList();
    }
    // Custom Method - Compare Students by Name
    public int compareStudentNames(Long id1, Long id2) {
        Student student1 = getStudentById(id1);
        Student student2 = getStudentById(id2);
        return student1.getStudentName()
                .compareTo(student2.getStudentName());
    }
}
```

StudentController.java

```
package com.example.student.controller;
import java.util.List;
import org.springframework.web.bind.annotation.*;
import com.example.student.model.Student;
import com.example.student.service.StudentService;
import jakarta.validation.Valid;
@RestController
@RequestMapping("/students")
public class StudentController {
  private final StudentService service;
  public StudentController(StudentService service) {
    this.service = service;
  }
  // POST - Register Student
  @PostMapping
  public Student registerStudent(@Valid @RequestBody Student student) {
    return service.registerStudent(student);
  }
  // GET - View All Students
  @GetMapping
  public List<Student> getAllStudents() {
    return service.getAllStudents();
  }
  // GET - Find Student By ID
  @GetMapping("/{id}")
  public Student getStudentById(@PathVariable Long id) {
    return service.getStudentById(id);
  }
```

```
// PUT - Update Complete Student Details
@PutMapping("/{id}")
public Student updateStudent(
      @PathVariable Long id,
      @Valid @RequestBody Student student) {
   return service.updateStudent(id, student);
}
// PATCH - Update Selected Fields
@PatchMapping("/{id}")
public Student partialUpdate(
      @PathVariable Long id,
      @RequestBody Student student) {
   return service.partialUpdate(id, student);
}
// DELETE - Delete Student
@DeleteMapping("/{id}")
public String deleteStudent(@PathVariable Long id) {
   return service.deleteStudent(id);
}
// Custom API - Search Student By Name
@GetMapping("/search")
public List<Student> searchByName(@RequestParam String name) {
   return service.searchByName(name);
}
// Custom API - Sort Students By Name
@GetMapping("/sort")
public List<Student> sortByName() {
   return service.sortByName();
```

```
    }
    // Custom API - Get Student Names
    @GetMapping("/names")
    public List<String> getStudentNames() {
        return service.getStudentNames();
    }
    // Custom API - Filter Students By Minimum Age
    @GetMapping("/age")
    public List<Student> getStudentsByMinimumAge(@RequestParam int age) {
        return service.getStudentsByMinimumAge(age);
    }
    // Custom API - Compare Two Students By Name
    @GetMapping("/compare")
    public int compareStudentNames(
            @RequestParam Long id1,
            @RequestParam Long id2) {
        return service.compareStudentNames(id1, id2);
    }
}
```

GlobalExceptionHandler.java

```
package com.example.student.exception;
import java.util.LinkedHashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestControllerAdvice;
```

```
@RestControllerAdvice
public class GlobalExceptionHandler {
  // Custom Exception
  @ExceptionHandler(StudentNotFoundException.class)
  @ResponseStatus(HttpStatus.NOT_FOUND)
  public Map<String, String> handleStudentNotFound(
      StudentNotFoundException ex) {
    return Map.of("message", ex.getMessage());
  }
  // Validation Exception
  @ExceptionHandler(MethodArgumentNotValidException.class)
  @ResponseStatus(HttpStatus.BAD_REQUEST)
  public Map<String, String> handleValidation(
      MethodArgumentNotValidException ex) {
    Map<String, String> errors = new LinkedHashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(error ->
      errors.putIfAbsent(
        error.getField(),
        error.getDefaultMessage()
      )
    );
    return errors;
  }
  // IllegalArgumentException
  @ExceptionHandler(IllegalArgumentException.class)
  @ResponseStatus(HttpStatus.BAD_REQUEST)
  public Map<String, String> handleIllegalArgument(
      IllegalArgumentException ex) {
```

```
    return Map.of("message", ex.getMessage());
  }
  // General Exception
  @ExceptionHandler(Exception.class)
  @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
  public Map<String, String> handleGeneral(Exception ex) {
    return Map.of(
      "message",
      "An unexpected error occurred"
    );
  }
}
```

StudentNotFoundException

```
package com.example.student.exception;
public class StudentNotFoundException extends RuntimeException {
  public StudentNotFoundException(String message) {
    super(message);
  }
}
```

PROJECT 3: ONLINE FOOD ORDER:

OnlineFoodOrderApplication.Java

```
package com.demo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import jakarta.validation.constraints.DecimalMin;
@SpringBootApplication
public class OnlineFoodOrderApplication {
	public static void main(String[] args) {
		SpringApplication.run(OnlineFoodOrderApplication.class, args);
	}
}
```

FoodOrder.java

```
package com.demo.model;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Data
@NoArgsConstructor
@AllArgsConstructor
public class FoodOrder {
  private Integer orderId;
  @NotBlank(message = "Customer name is required")
  private String customerName;
  @NotBlank(message = "Food item cannot be empty")
```

```
    private String foodItem;

    @Min(value = 1, message = "Quantity must be at least 1")

    private Integer quantity;

    @NotNull(message = "Price is mandatory")

    @DecimalMin(value = "0.0", inclusive = false, message = "Price must be greater than zero")

    private Double price;

}
```

FoodOrderService.java

```
package com.demo.service;

import java.util.ArrayList;

import java.util.Comparator;

import java.util.List;

import java.util.stream.Collectors;

import org.springframework.stereotype.Service;

import com.demo.exception.OrderNotFoundException;

import com.demo.model.FoodOrder;

@Service

public class FoodOrderService {

    private List<FoodOrder> orders = new ArrayList<>();

    private int nextId = 1;

    // POST - Add Order

    public FoodOrder addOrder(FoodOrder order) {

        order.setOrderId(nextId++);

        orders.add(order);

        return order;

    }

    // GET - Get All Orders
```

```
public List<FoodOrder> getAllOrders() {
   return orders;
}
// GET - Get Order By ID
public FoodOrder getOrderById(int id) {
   if (id <= 0) {
      throw new IllegalArgumentException(
         "Order ID must be greater than zero");
   }
   return orders.stream()
         .filter(o -> o.getOrderId() == id)
         .findFirst()
         .orElseThrow(() ->
            new OrderNotFoundException(
               "Order not found with ID: " + id));
}
// PUT - Update Complete Order
public FoodOrder updateOrder(int id, FoodOrder newOrder) {
   FoodOrder oldOrder = getOrderById(id);
   oldOrder.setCustomerName(newOrder.getCustomerName());
   oldOrder.setFoodItem(newOrder.getFoodItem());
   oldOrder.setQuantity(newOrder.getQuantity());
   oldOrder.setPrice(newOrder.getPrice());
   return oldOrder;
}
// DELETE - Delete Order
public String deleteOrder(int id) {
   FoodOrder order = getOrderById(id);
```

```
    orders.remove(order);
    return "Order deleted successfully";
}
// PATCH - Update Quantity Only
public FoodOrder updateQuantity(int id, int quantity) {
    if (quantity < 1) {
        throw new IllegalArgumentException(
            "Quantity must be at least 1");
    }
    FoodOrder order = getOrderById(id);
    order.setQuantity(quantity);
    return order;
}
// Custom Method - Search Food By Partial Name
public List<FoodOrder> searchByFoodItem(String name) {
    return orders.stream()
            .filter(o -> o.getFoodItem()
                    .toLowerCase()
                    .contains(name.toLowerCase()))
            .collect(Collectors.toList());
}
// Custom Method - Filter Orders By Minimum Price
public List<FoodOrder> filterByMinimumPrice(double price) {
    return orders.stream()
            .filter(o -> o.getPrice() >= price)
            .collect(Collectors.toList());
}
// Custom Method - Sort Orders By Price
```

```
    public List<FoodOrder> sortByPrice() {
        return orders.stream()
                .sorted(Comparator.comparing(FoodOrder::getPrice))
                .collect(Collectors.toList());
    }
    // Custom Method - Get Food Item Names Using map()
    public List<String> getFoodItemNames() {
        return orders.stream()
                .map(FoodOrder::getFoodItem)
                .collect(Collectors.toList());
    }
    // Custom Method - Calculate Total Order Amount
    public double calculateTotalAmount(int id) {
        FoodOrder order = getOrderById(id);
        return order.getPrice() * order.getQuantity();
    }
}
```

FoodOrderController.java

```
package com.demo.controller;
import java.util.List;
import org.springframework.web.bind.annotation.*;
import com.demo.model.FoodOrder;
import com.demo.service.FoodOrderService;
import jakarta.validation.Valid;
@RestController
@RequestMapping("/orders")
public class FoodOrderController {
    private final FoodOrderService service;
```

```
public FoodOrderController(FoodOrderService service) {
   this.service = service;
}
// POST - Add Order
@PostMapping
public FoodOrder addOrder(@Valid @RequestBody FoodOrder order) {
   return service.addOrder(order);
}
// GET - Get All Orders
@GetMapping
public List<FoodOrder> getAllOrders() {
   return service.getAllOrders();
}
// GET - Get Order By ID
@GetMapping("/{id}")
public FoodOrder getOrderById(@PathVariable int id) {
   return service.getOrderById(id);
}
// PUT - Update Complete Order
@PutMapping("/{id}")
public FoodOrder updateOrder(
      @PathVariable int id,
      @Valid @RequestBody FoodOrder order) {
   return service.updateOrder(id, order);
}
// DELETE - Delete Order
@DeleteMapping("/{id}")
public String deleteOrder(@PathVariable int id) {
```

```
    return service.deleteOrder(id);
}
// PATCH - Update Quantity Only
@PatchMapping("/{id}/quantity")
public FoodOrder updateQuantity(
        @PathVariable int id,
        @RequestParam int quantity) {
    return service.updateQuantity(id, quantity);
}
// Custom API - Search Food By Name
@GetMapping("/search")
public List<FoodOrder> searchByFoodItem(
        @RequestParam String name) {
    return service.searchByFoodItem(name);
}
// Custom API - Filter By Minimum Price
@GetMapping("/filter")
public List<FoodOrder> filterByMinimumPrice(
        @RequestParam double price) {
    return service.filterByMinimumPrice(price);
}
// Custom API - Sort By Price
@GetMapping("/sort")
public List<FoodOrder> sortByPrice() {
    return service.sortByPrice();
}
// Custom API - Get Food Names
@GetMapping("/food-names")
```

```
    public List<String> getFoodItemNames() {
        return service.getFoodItemNames();
    }
    // Custom API - Calculate Total Amount
    @GetMapping("/{id}/total")
    public double calculateTotalAmount(@PathVariable int id) {
        return service.calculateTotalAmount(id);
    }
}
```

OrderNotFoundException

```
package com.demo.exception;
public class OrderNotFoundException extends RuntimeException {
    public OrderNotFoundException(String message) {
        super(message);
    }
}
```

GlobalExceptionHandler

```
package com.demo.exception;
import java.util.HashMap;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import
org.springframework.web.method.annotation.MethodArgumentTypeMismatchExceptio
n;
```

```
@RestControllerAdvice
public class GlobalExceptionHandler {
  // Custom Exception
  @ExceptionHandler(OrderNotFoundException.class)
  public ResponseEntity<String> handleOrderNotFound(
      OrderNotFoundException ex) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND)
        .body(ex.getMessage());
  }
  // Validation Errors
  @ExceptionHandler(MethodArgumentNotValidException.class)
  public ResponseEntity<Map<String, String>> handleValidation(
      MethodArgumentNotValidException ex) {
    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(error ->
      errors.put(error.getField(), error.getDefaultMessage())
    );
    return ResponseEntity.badRequest().body(errors);
  }
  // IllegalArgumentException
  @ExceptionHandler(IllegalArgumentException.class)
  public ResponseEntity<String> handleIllegalArgument(
      IllegalArgumentException ex) {
    return ResponseEntity.badRequest()
        .body(ex.getMessage());
  }
// Type Mismatch Exception
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
```

```
public ResponseEntity<String> handleTypeMismatch(
      MethodArgumentTypeMismatchException ex) {
   return ResponseEntity.badRequest()
         .body("Invalid ID. Please enter a number.");
}
   // General Exception
   @ExceptionHandler(Exception.class)
   public ResponseEntity<String> handleGeneralException(
         Exception ex) {
      ex.printStackTrace();
      return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Error: " + ex.getMessage());
   }}
```